

Dev Workflows

EAS Build · OTA Updates · Railway · Git · Secrets

Djiguitech — Social Seller SaaS

Ce module couvre les workflows de developpement et de deploiement du projet Social Seller : comment un changement de code arrive en production, les deux chemins (backend vs mobile), le choix EAS Build vs OTA Updates, et les bonnes pratiques autour des secrets.

1. Vue d'ensemble de la stack

Le projet est decoupe en deux couches deployees independamment :

Couche	Technologie	Deploye sur	Declenche par
Backend (API)	Node.js / Express / TypeScript	Railway	git push sur main
Frontend mobile	Expo SDK 56 / React Native	EAS (Expo Application Services)	eas build ou eas update
Base de donnees	Supabase (PostgreSQL + RLS)	Supabase Cloud	Migrations SQL manuelles
Webhooks Meta	Callbacks WhatsApp / Facebook	Railway (meme serveur backend)	Automatique

INFO Backend et frontend ont des cycles independants. Un push backend deploie en ~2 min. Un changement frontend necessite soit un build complet (~15 min) soit une update OTA (~2 min).

2. Workflow Backend — Railway

Railway est connecte a la branche main du repo GitHub. Chaque push sur main declenche automatiquement un redéploiement.

1**Modifier le code**

Fichiers dans backend/src/routes/, backend/src/services/, etc.

2**Verifier TypeScript**

cd backend && npx tsc --noEmit — zero erreur avant de committer

3

Lancer les tests

cd backend && npx vitest run — tous les tests doivent passer

4

Commit conventionnel

git commit -m 'feat(US-XX): description'

5

Push vers main

git push — Railway detecte le push et redéploie automatiquement

6

Verifier les logs Railway

Dashboard Railway → Deployments → logs en temps reel

Convention de commit

```
feat(US-12): resoudre conversation depuis le header fix(webhook): ajouter tenant_id au  
filtre delivery status refactor(orders): extraire notifyClientOnStatusChange chore:  
mettre a jour les dependances npm
```

ATTENTION Ne JAMAIS committer de cle API, token ou secret. Les variables sensibles vivent dans .env (exclu de git). Railway injecte les secrets via ses variables d'environnement.

3. Workflow Frontend Mobile — EAS

Deux modes de déploiement selon la nature du changement :

	EAS Build (natif)	EAS Update (OTA)
Duree	~15-20 minutes	~2 minutes
Produit	APK / AAB / IPA	Bundle JS uniquement
Installation	Telechargement manuel	Automatique au demarrage
Quand	Changement natif obligatoire	JS/TS/assets seulement
Cout	Credits EAS (limites)	Gratuit (illimite)
Commande	<code>eas build --platform android --profile preview</code>	<code>eas update --branch preview</code>

EAS Build obligatoire quand :

- Ajout d'un plugin natif Expo (expo-sharing, expo-camera, expo-file-system...)
- Modification de app.config.ts (permissions, plugins, scheme, package name...)
- Modification de eas.json
- Mise a jour du SDK natif Expo (ex: SDK 56 vers 57)
- Changement du package Android (com.djiguitech.socialseller)

OTA Update suffit quand :

- Modification de composants React Native (.tsx, .ts, .js)
- Ajout ou modification de screens, logique metier, styles
- Corrections de bugs purement JavaScript/TypeScript
- Changement de textes, couleurs, icones (assets bundless)

IMPORTANT L'OTA fonctionne uniquement si le runtimeVersion de l'update correspond a celui du build installe. Avec la policy 'appVersion', tous les builds d'une meme version sont compatibles.

4. Configuration OTA — app.config.ts et eas.json

Trois elements obligatoires dans app.config.ts :

```
export default ({ config }): ExpoConfig => ({ ...config, // 1. URL du service EAS Update
updates: { url: 'https://u.expo.dev/022930f8-...', }, // 2. Politique de compatibilite
build <-> update runtimeVersion: { policy: 'appVersion', // build v1.0.0 accepte updates
v1.0.0 }, // 3. Plugin expo-updates dans la liste plugins: ['expo-router',
'expo-secure-store', 'expo-updates'], })
```

Le champ channel dans eas.json :

```
{ "build": { "preview": { "distribution": "internal", "channel": "preview", // associe ce
build au canal OTA preview "env": { "EXPO_PUBLIC_API_URL": "https://..." } },
"production": { "autoIncrement": true, "channel": "production", "env": { ... } } } }
```

ATTENTION Sans le champ channel dans eas.json, les updates OTA publiees ne sont jamais recues par le build. Sans expo-updates dans les plugins, le build ne peut pas recevoir d'updates.

Commandes essentielles

```
# Build APK (distributable directement par APK) eas build --platform android --profile
preview # Publier une mise a jour OTA eas update --branch preview --message 'fix:
correction bug inbox' # Verifier les updates disponibles eas update:list # Verifier les
builds eas build:list
```

5. Gestion des secrets

Type de secret	Stockage	Methode d'injection
Variables backend (DB, JWT...)	Railway Dashboard	Environment Variables → process.env.*
Variables frontend publiques	eas.json > env	EXPO_PUBLIC_* → accessibles dans l'app
Tokens API (WhatsApp, Facebook)	Supabase (chiffres AES-256-GCM)	Dechiffres a la volée par le backend
Tokens de session utilisateur	expo-secure-store	Jamais AsyncStorage — chiffre par l'OS

Chiffrer un token API

```
# Generer une cle de chiffrement 32 bytes node -e
"console.log(require('crypto').randomBytes(32).toString('base64'))" # Chiffrer un token
(script standalone sans imports projet) TOKEN_ENCRYPTION_KEY=<cle_base64>
TOKEN=<mon_token> \ npx ts-node --skip-project backend/scripts/encrypt-token.ts #
Resultat : \x<hex> a coller dans Supabase colonne access_token_enc
```

INFO Le script encrypt-token.ts est volontairement standalone (pas d'imports du projet) pour eviter que la validation des variables Railway ne bloque son execution en local. Ne jamais committer ce script avec un vrai token dedans.

6. Checklist de deployment

Avant chaque push backend

- [] `npx tsc --noEmit` depuis le dossier backend — zero erreur TypeScript
- [] `npx vitest run` depuis le dossier backend — tous les tests passent
- [] Aucune cle API ou secret hardcode dans le code
- [] Toutes les requetes Supabase filtrent par `tenant_id`
- [] Les nouveaux endpoints sont proteges par `requireAuth + authenticatedLimiter`
- [] Les inputs utilisateur sont valides via zod avant tout traitement

Avant chaque EAS Build

- [] `npx tsc --noEmit` — zero erreur TypeScript
- [] `app.config.ts` : `updates.url`, `runtimeVersion` et `expo-updates` presents
- [] `eas.json` : channel defini dans le profil cible (preview ou production)
- [] Variables `EXPO_PUBLIC_*` a jour dans `eas.json`
- [] Version incrementee si besoin (`autoIncrement: true` en production)

Avant chaque EAS Update (OTA)

- [] Changement purement JS/TS — aucun plugin natif ajoute
- [] `app.config.ts` non modifie (si modifie → build obligatoire)
- [] Bon `--branch` specifie (preview vs production)
- [] Message descriptif : `eas update --message 'feat: ...'`
- [] Apres publication : redemarrer l'app 2x sur le device pour forcer la reception

7. Erreurs courantes et solutions

Probleme : OTA publiee mais app ne se met pas a jour	Solution : Verifier les 3 elements : (1) expo-updates dans app.config.ts plugins, (2) channel dans eas.json, (3) updates.url et runtimeVersion dans app.config.ts. Si l'un manquait dans le build installe, un nouveau build est necessaire.
Probleme : WhatsApp 401 Unauthorized	Solution : Le token d'accès Meta a expiré. Les tokens temporaires durent 24h. Solution permanente : System User Token sans expiration via Meta Business Manager. Solution immédiat : nouveau token + script encrypt-token.ts + mise à jour SQL Supabase.
Probleme : WhatsApp/API 409 Conflict	Solution : La connexion sociale a disconnected_at non null en base. La requête de mise à jour filtre WHERE disconnected_at IS NULL et touche 0 lignes. Solution : reactiver la connexion (UPDATE social_connections SET disconnected_at = NULL) avant de mettre à jour le token.
Probleme : git: fatal HEAD.lock / index.lock exists	Solution : Un processus git a crashé et laisse un verrou. Solution : rm .git/HEAD.lock && rm .git/index.lock puis relancer la commande git.
Probleme : EAS build genere un AAB au lieu d'un APK	Solution : Le profil production produit un AAB (Android App Bundle) pour le Play Store. Pour un APK installable directement, utiliser le profil preview.
Probleme : Tests echouent : Required sur les variables d'environnement	Solution : Les tests sont lancés depuis la racine du projet (npm test) au lieu du dossier backend. Toujours lancer : cd backend && npx vitest run.
Probleme : SQL UPDATE touche 0 lignes (0 rows affected)	Solution : Toujours vérifier que la requête cible les bonnes colonnes et valeurs avant exécution. Utiliser d'abord un SELECT avec les memes conditions WHERE pour confirmer que les lignes existent bien avant de lancer l'UPDATE.